FRC RAG Application Documentation

Section 1 – Quick links:

- GitHub Repository
- RAGAS Results

Section 2 – Documentation:

- JS Documentation:
  - *POST - /api/query*
    - Sends request with user information to the Python route: **/api/query**
  - *GET - /*
    - Renders HTML Frontend
- Python Documentation (Models):
  - **Embedding Model:** MiniLM-L6-v2
  - **User-Uploaded Image Processing Model:** Bakllava
  - **Decomposition and Rephrasing Model:** Llama 3.2:3b
  - **Sentiment Classifier:** Llama 3.2:3b
  - **Intent Classifier:** Llama 3.2:3b
  - **User Response Generator:** Qwen 2.5:7b
- Python Documentation (Functions):
  - clean_ocr_to_markdown_table(raw_ocr_text):
    - Takes Raw CSV text from the vision model and formats it into a well-formed markdown table that marked.js can render.
  - perform_vision_ocr(base64_image_string):
    - Leverages a local vision model to accurately parse text, tables, or diagram rules out of user-uploaded images.
  - decompose_and_rephrase_query(user_query):
    - Decomposes compound questions into subqueries and rephrases them to maximize vector database embedding matches
  - extract_pdf_with_tables(pdf_path):
    - Extracts raw text and table structures from ALL pages of the PDF, loops page-by-page with try-except blocks to prevent errors

- chunk_text(text, chunk_prefix = ""):
  - Splits the chunk into coherent blocks while preserving table structure. Uses delimiters such as newlines, double newlines, periods, and spaces.
- load_chunks_to_model(chunks, prefix_id, source_type, batch_size=100):
  - Vectorizes chunks using unique prefixed tracking strings.
  - The prefixed tracking strings indicate the source the text belongs to.
- generate_database():
  - Build the database using the manual and team updates.
- classify_query_sentiment(user_query):
  - Uses a lightweight model to quickly categorize the user's emotional tone before prompt generation
  - *Sentiment Response Mapping:*
    - **Neutral:** No Prefix
    - **Anger:** Apology
    - **Frustration:** Validation
    - **Confusion:** Reassurance
    - **Sarcasm:** Assurance of data reliability
    - **Sadness:** Provides encouragement to fix the issue
    - **Satisfaction:** Provides positive reinforcement

- Python Documentation (Routes):
  - *GET - /*
    - Returns App Status
  - *POST - /api/query*

1. Stores User Query, Chat History, and Image Bytes
2. Performs OCR on user-uploaded images using perform_vision_ocr(base64_image_string)
3. Detects User Sentiment using classify_query_sentiment(user_query)
4. Detects User Intent to prevent prompt injection
   i. If Prompt injection is detected, it displays a fallback answer and ends the execution sequence
5. Decomposes and rephrases the query using decompose_and_rephrase_query(user_query)
6. Embeds all search queries and returns the top 10 results from the Game Manual and the top 8 results from the Team Updates
7. Checks for terms such as "Team Update" or "Update" to boost results from team update documents for better retrieval.
8. Adds Newest Results to retrieved contexts

9. Re-ranks context based on chunks with a cosine similarity of 0.1 relative to the user prompt
    i. The query and the embeddings are first converted to tensors, then normalized, and the cosine similarity is computed via matrix multiplication.
10. Generates the text output using an LLM and displays the LLM Response

Section 3 – Payload Details:

- ## Port Configuration:
    - **JS Backend (Node.js/Express.js):** Port 3000 (localhost:3000)
    - **Python Backend (Flask):** Port 5000 (localhost:5000)
- ## Example Payloads/Responses:

GET - /

```
JSON
{
  "status": "ok",
  "indexed_chunks": 1240
}
```

POST - /api/query (payload) ***MAX SIZE: 50mb***

```
JSON
{
  "query": "Is my robot allowed to extend past 12 inches per G101?",
  "chat_history": [],
  "image": "... (optional)"
}
```

POST - /api/query (response)

```
JSON
{
  "answer": "It makes total sense that this is unclear... According to G101...",
  "retrieved_chunks": ["...", "..."],
  "detected_sentiment": "CONFUSION",
  "sub_queries_generated": ["Robot extension limit rule G101", "FRC G101 extension rule limit"]
}
```